

Aether Delivery Decisions Log

This file records durable, cross-session engineering and product decisions for the Aether delivery effort. Each entry is append-only. Newest entries at the top.

D-0001 — Delivery approach: vertical slices, strict TDD, single source of truth

Date: 2026-07-01 · **Author:** Aether Delivery Agent (Session 1) · **Status:** Adopted

Context. The repository currently holds architecture, design, research, and 16 high-fidelity HTML wireframes. The design `review_report.md` flagged navigation inconsistency (a BLOCKER), cross-screen data contradictions, and missing safety rails. Implementation has not started.

Decision.

1. **Vertical slices.** All work is decomposed into small, independently shippable slices with a stable ID (e.g. `P0-S01`). Each slice has a Given/When/Then acceptance statement, is tracked in `PROGRESS.md`, and lands in its own conventional commit tagged with the slice ID.
2. **Strict TDD for code.** Every code slice follows RED → GREEN → (REFACTOR). A failing test is written and observed to fail before the implementation is written. Wireframe/design slices are verified by structural checks and manual visual review rather than unit tests.
3. **Single source of truth.** `docs/delivery/PROGRESS.md` is the canonical status ledger. `design/DESIGN.md` is the canonical design language. `design/review_report.md` tracks audit findings and their resolution. This repo is the single source of truth across sessions.
4. **Branching.** Feature work happens on task branches (Phase 0 wireframes on `phase-0/wireframes`). `main` is never edited directly. Changes reach `main` only via PR.
5. **Secrets.** `.env` is never committed. Only `.env.example` (keys with placeholder values) is version-controlled. No real API key is ever logged or printed.

Consequences. Slower per-commit but fully auditable, resumable across sessions, and safe.

D-0002 — Standardize on the 12-item sidebar (Schema A) across every screen

Date: 2026-07-01 · **Author:** Aether Delivery Agent (Session 1) · **Status:** Adopted

Context. `review_report.md` identifies two competing navigation schemas. Schema A (12 items: Dashboard, Jobs, Resume Studio, Story Bank, Applications, Interview Center, Networking, Email Center, Agents, Analytics, Offers, Settings) is used by 10 screens; Schema B (9 items, adds a phantom “Cover Letters”, drops 4 real screens) is used by 4 screens. This is a BLOCKER.

Decision. Every screen — existing and newly created — uses the identical Schema A 12-item sidebar. The active item is highlighted with the coral accent (`#FF6B35`). The logo lockup and bottom profile widget are standardized. The label text “Resume Studio” is used verbatim on all screens to guarantee pixel-identical navigation (avoids mixing “Resume” / “Résumé”).

Consequences. New screens created this session (Onboarding Wizard, Cover Letter Studio, Notification Center) ship with Schema A. Screens touched for other slices have their sidebar reconciled to Schema A opportunistically.

D-0003 — Canonical funnel numbers

Date: 2026-07-01 · **Author:** Aether Delivery Agent (Session 1) · **Status:** Adopted

Context. `review_report.md` found three different funnel datasets across Dashboard, Tracker Sankey, and Analytics. The delivery brief specifies a single canonical funnel.

Decision. The canonical application funnel is:

847 (Jobs Found) → 412 (Applied) → 156 (Screened) → 23 (Interviewed) → 4 (Offers) .

These exact numbers must be used everywhere funnel data appears. Analytics is aligned first (P0-S05); Dashboard and Application Tracker will be reconciled in follow-up slices.

Consequences. Any screen showing funnel data must reference this set. Deviations are bugs.

Amended by D-0028 (2026-07-13). The numbers above are the canonical design-time example used to validate layout/consistency, not a literal string every live screen must render. See D-0028 for the corrected scope: every screen must compute the funnel from the same live query (already true), but a screen showing correct real per-user data that differs from 847/412/156/23/4 is not a bug.

D-0004 — Test harness & CI toolchain

Date: 2026-07-01 · **Author:** Aether Delivery Agent (Session 1) · **Status:** Adopted

Context. The implementation guide specifies a pnpm + Turborepo monorepo with a Next.js 14 `apps/web` frontend and a FastAPI `apps/api` backend, tested in GitHub Actions.

Decision. For P1-S00 we bootstrap a minimal but runnable harness aligned to the guide:

- `apps/web` : TypeScript + **Vitest** for unit tests (`__tests__/health.test.ts`).
- `apps/api` : Python 3.11 + **pytest** (`tests/test_health.py`).
- Root: pnpm workspace (`pnpm-workspace.yaml`), shared scripts.
- CI (`ci/github-actions-ci.yml`): install → lint → type-check → unit tests → build, on

Node 20 and **Python 3.11**, with dependency caching.

Consequences. Later slices extend these apps rather than re-scaffolding. Vitest chosen over Jest for speed and zero-config ESM/TS support; it is swappable without changing slice contracts.

Update (D-0005 cross-ref). The CI workflow is stored at `ci/github-actions-ci.yml` (an inert template) rather than `.github/workflows/ci.yml`, so branches push and merge to `main` without requiring the GitHub App `workflows` permission. Activation instructions live in `ci/README.md`. This keeps CI-CD simple with no permission friction; the workflow is copied into `.github/workflows/` only when the team chooses to switch CI on.

D-0006 — Auth: framework-free JWT session layer now, NextAuth route wiring in P1-S06

Date: 2026-07-02 · **Author:** Aether Delivery Agent (Session 2) · **Status:** Adopted

Context. P1-S03 delivers authentication, but Next.js itself is not introduced until the dashboard shell (P1-S06). NextAuth.js requires `next / react` as peer dependencies and its route handler cannot run without a Next.js app. Pulling those heavy peers a slice early — purely to host a handler that has nothing to serve yet — would add fragility with no functional payoff.

Decision. In P1-S03 we implement the security-critical core as framework-agnostic, fully-unit-tested TypeScript in `apps/web/src/lib/auth/` :

- `jwt.ts` — sign/verify session tokens via `jose` (the same library NextAuth uses internally), so the token format matches what NextAuth will issue later.
- `session.ts` — session model + forgiving token→session resolution.
- `require-auth.ts` — a `requireAuth` guard that reads a Bearer header or the session cookie and returns a discriminated result; works with the `Fetch Request` used by Next.js route handlers / middleware and with plain test doubles.
- `credentials.ts` — `authorizeCredentials`, the Credentials-provider callback, written with its data-access and password-verification dependencies injected (wired to `UserRepository` + a real hash comparison in P1-S06). The returned user never carries the password hash.
- `options.ts` — `authConfig`, a NextAuth-shaped config object (Credentials provider + stateless JWT session strategy) ready to hand to `NextAuth(authConfig)`.
- `test-helpers.ts` — token/session factories shared by unit tests and future E2E setup.

Consequences. The auth contract is provable without booting a framework and the suite stays green offline. When P1-S06 adds Next.js, the NextAuth route handler (`app/api/auth/[...nextauth]/route.ts`) simply consumes `authConfig` + the `jose` helpers and the Credentials `authorize` delegates to `authorizeCredentials`. Signing secret is read from `NEXTAUTH_SECRET` (already in `.env.example`) and is never logged.

D-0007 — Web: App Router + offline-safe fonts via `<link>` (not `next/font`)

Date: 2026-07-02 · **Author:** Aether Delivery Agent (Session 2) · **Status:** Adopted

Context. P1-S06 introduces Next.js for the dashboard shell. Two choices needed pinning: (1) how to load the Inter / JetBrains Mono web fonts and Font Awesome icon set, and (2) how the NextAuth route handler deferred in D-0006 gets wired.

Decision.

- **App Router** (`src/app/*`) is the routing model. The 12-item Schema-A sidebar renders straight from a single pure-data contract, `src/lib/navigation.ts` (no React/Next imports), so the ordering is unit-testable under a plain Node/Vitest environment and is asserted by `__tests__/navigation.test.ts`.
- **Fonts/icons load via `<link>` tags in the root layout `<head>`**, not `next/font`. `next/font` fetches font files from Google at build time; that would make `next build` (and therefore CI and offline builds) network-dependent and non-deterministic. Loading via `<link>` keeps builds hermetic. Because this is the correct App Router pattern (there is no Pages-Router `_document.js`), the legacy `@next/next/no-page-custom-font` lint rule is disabled in `.eslintrc.json` with this rationale.

- **NextAuth wiring (fulfils D-0006).** `src/lib/auth/next-auth-options.ts` builds a real `NextAuthOptions` (Credentials provider + stateless-JWT session) whose `authorize` delegates to the P1-S03 `authorizeCredentials`. `src/app/api/auth/[...nextauth]/route.ts` exports the `NextAuth` handler as both `GET` and `POST`. The user-lookup / password-verify dependencies are placeholders returning `null` / `false` (no user store is seeded yet); they are wired to `UserRepository` + a real hash comparison in Phase 2. Secret is read from `NEXTAUTH_SECRET`; never logged.

Alternatives. `next/font` (rejected: build-time network); embedding the sidebar order in the component (rejected: not independently testable); pulling NextAuth peers back in P1-S03 (rejected in D-0006).

Consequences. `next build`, `type-check`, `lint`, Vitest (25 web tests) and a Playwright smoke test (`e2e/dashboard.spec.ts`, 2 tests asserting the 12 nav items render and `/ → /dashboard`) all pass offline. Build tooling added to `@aether/web`: `next`, `react`, `react-dom`, `next-auth`, `tailwindcss` / `postcss` / `autoprefixer`, `@types/*`, `eslint-config-next`, `@playwright/test`. The `build` script is now `next build` (the orphaned `tsconfig.build.json` and `.eslintrc.cjs` were removed). `unrs-resolver` was added to `pnpm-workspace.yaml`'s allowed build scripts (a native dependency of `eslint-config-next`). **Reversible?** Yes — fonts could later move to `next/font` with a self-hosted/offline cache without touching the navigation contract or auth wiring.

D-0008 — Activate CI at `.github/workflows/ci.yml` (mirror retained; staged push)

Date: 2026-07-02 · **Author:** Aether Delivery Agent (Session 2) · **Status:** Adopted

Context. Phase 1 needs a live CI gate. Through Phase 0 the workflow was parked as an inert template at `ci/github-actions-ci.yml` (D-0004) because the Abacus GitHub App is not guaranteed the

`workflows` permission, and a push that touches `.github/workflows/**` is rejected without it. Slice P1-S11 also introduced offline LLM fixtures that CI must exercise deterministically.

Decision.

- **Activate** the pipeline at `.github/workflows/ci.yml`, and keep `ci/github-actions-ci.yml` as a **verbatim mirror** (kept identical on every edit via `cp`). The mirror guarantees the exact workflow stays reviewable/tracked even if the workflow push is rejected.

- **Pipeline shape.** Four PR/push jobs — `security` (fails if `.env` is tracked or a real-looking `sk-or-v1-<32+ alnum>` key appears in source), `node` (Node 20: `install` → `@aether/db` `prisma:generate` → `lint` → `type-check` → `unit tests` → `build`, all recursive across the workspace, with `AETHER_LLM_MODE=replay`), `api` (Python 3.11: `ruff` → `mypy` → `pytest`), and `e2e` (Playwright chromium smoke, `needs: node`). One schedule/dispatch-only job — `live-openrouter` — is `continue-on-error` (non-blocking), skips when `OPENROUTER_API_KEY` is unset, and never gates PRs.

- **Prisma generate precedes type-check/build** because the web app and repositories type-check against `@prisma/client`; without a generated client the Node job would fail.

- **Key-scan regex requires a long tail** (`{32,}`) so clearly-synthetic placeholders (the `redactSecrets` unit test's short `sk-or-v1-abcdef...`, and the fixtures README's `sk-...`) are not false-positives, while real 64-hex keys are still caught. The pattern does not match itself in the workflow file (it is followed by `[]`).

- **Push structure (workflows permission).** The Abacus GitHub App installation lacks the `workflows` permission, so GitHub rejects any push that creates/updates a file under `.github/workflows/**` (confirmed empirically — the first attempt to push a commit containing `.github/workflows/ci.yml` was `remote rejected`). Crucially, the byte-identical **mirror** `ci/github-actions-ci.yml` lives

outside `.github/workflows/` and pushes without any special permission. The slice is therefore split so everything lands on the remote except the one genuinely-blocked file:

1. `f109757` — the full pipeline as the tracked mirror `ci/github-actions-ci.yml` + `ci/README.md`.
2. a `docs(progress)` commit recording the slice (+ this ADR + the Phase 1 self-review).
3. a final commit that drops the identical file into `.github/workflows/ci.yml` to activate it.

Commits 1-2 are pushed normally; commit 3 is the only thing that can be rejected. **To finish**

activation, either grant the app the `workflows` permission

(https://github.com/apps/abacusai/installations/select_target (https://github.com/apps/abacusai/installations/select_target)) and re-push, or copy

`ci/github-actions-ci.yml` into `.github/workflows/ci.yml` via the GitHub web editor (a UI commit is authored as the user, not the app, so no extra permission is needed). See `ci/README.md`.

Alternatives. Keep CI inert (rejected: Phase 1 wants a live gate); split web from packages into separate jobs (rejected: one recursive Node job is simpler and covers every package's tests, which the old web-only job missed); add a coverage-threshold gate now (deferred: `@vitest/coverage-v8` is not yet

a dependency — wiring coverage + per-package thresholds is its own slice, and adding an unconfigured

gate would break the green pipeline).

Consequences. Every gate was verified locally before commit: recursive lint / type-check / build exit 0, 63 unit tests green, API 22 pytest + ruff + mypy clean, Playwright smoke green, and both security checks behave correctly. Coverage enforcement (directive target $\geq 85\%$ on

`agents / db / shared` + API handlers) remains a tracked follow-up. **Reversible?** Yes — the workflow can be reverted to inert-template-only by deleting `.github/workflows/ci.yml`; the mirror and harness remain.

D-0009 — Graceful placeholder + pathname-aware sidebar for unbuilt dashboard routes (P1-S12)

Context. Deploying Phase 1 exposed a UX gap: the 12-item Schema-A sidebar links to a route per section, but only `/dashboard` has a page (the feature workspaces are later phases). On the live deployment, clicking any of the other 11 nav items returned a bare Next.js 404, and the dashboard layout hard-coded `activeHref="/dashboard"` (a TODO deferred in P1-S06), so the active highlight never

tracked the current route.

Decision. Harden the shell so it degrades gracefully instead of shipping dead links:

- Add a pure `findNavItemByHref(href)` resolver in `apps/web/src/lib/navigation.ts` — prefix-based, most-specific-match-wins, returning `undefined` for a path that maps to no known section. It is React/Next-free so it unit-tests in plain Node (as the rest of the nav contract does).
- Make `Sidebar` a client component that reads `usePathname()` and highlights the owning section on any route (`activeHref` remains an optional override for tests/stories). The dashboard layout no longer passes a hard-coded value — this closes the P1-S06 “active item resolved in a later slice” TODO.
- Add a catch-all `app/dashboard/[...slug]/page.tsx` that renders the resolved section title inside the existing shell with an honest “planned for a later phase” panel; unknown routes fall back to a generic placeholder. Every route now returns 200, not 404.

Alternatives. (a) Leave the 404s and document them (rejected: a deployed foundation demo where 11/12 nav clicks 404 misrepresents readiness and invites reviewer noise). (b) Build stub feature pages (rejected: that is later-phase scope and risks fabricating functionality the wireframes imply but Phase 1 does not yet deliver — the placeholder is explicit that the workspace is not built). (c) A single `not-found.tsx` (rejected: it cannot title the section or keep the correct nav active, and still reads as an error rather than a roadmap state).

Consequences. +4 resolver unit tests (web 25 → 29) and +1 Playwright smoke; full suite green; verified on the live deployment (all 12 nav routes 200). The placeholder is deliberately honest about scope, so it does not overstate Phase 1. **Reversible?** Yes — deleting the catch-all route restores the prior behaviour; the resolver is additive and independently useful for future active-nav needs.

Amended by D-0032 (2026-07-13). The generic placeholder pattern here covers every unbuilt route mechanically, but Interview Center specifically now has its own named deferral ADR (D-0032), mirroring how D-0025/D-0026/D-0027 name their deferrals, so its Phase 3+ status is discoverable without reading a code comment.

D-0010 — Cover letters live on the `Application` row (no new Prisma model)

Date. 2026-07-02 (P2-S07)

Context. The Prisma schema has no `CoverLetter` model, but P2-S07 needs to persist drafts, list them, and route them through the approval gate. Adding a model means a schema migration on the shared hosted Postgres mid-phase.

Decision. Store each draft on an `Application` row (`coverLetter` column, `status=draft`); the cover-letter id is the application id. `ApprovalRequest.type` only enumerates `application_submit` | `email_send` | `offer_response` , so cover-letter approvals use `application_submit` with `payload.kind = "cover_letter"` as the discriminator.

Alternatives. (a) New `CoverLetter` model + migration (rejected for now: schema churn mid-slice on a shared DB; the `Application` row is the natural owner since a letter exists only in service of an application). (b) Extend the `ApprovalType` enum (rejected: Postgres enum migration for a value the payload can express losslessly).

Consequences. Zero schema changes; drafts appear in both `/cover-letters` and the applications kanban (`draft` column) for free. **Reversible?** Yes — a later `CoverLetter` model can be backfilled from `Application.coverLetter` .

D-0011 — Record-replay LLM client; CI and tests never call a model

Date. 2026-07-02 (P2-S04..S08)

Context. Fit scoring, tailoring, cover letters and story extraction all need LLM output, but tests must be deterministic, free, and offline; and generated claims must never outrun the resume.

Decision. `LLMClient` with `AETHER_LLM_MODE=record|replay` (default **replay**): `replay` reads fixtures from `apps/api/tests/fixtures/llm/<prompt_name>/<key>.json`; `record` mode (opt-in, live key) writes them. Model tiers resolve from `AETHER_MODEL_<TIER>` env vars. Fixture content policy: every number/entity in a fixture must exist in the real base resume, because the fabrication guard and token-subset validators run on replayed output exactly as they would on live output.

Alternatives. Mock at the HTTP layer per test (rejected: N copies of the same canned payloads, and no path to “flip one env var and go live”). Skip validation in tests (rejected: the guard is the product).

Consequences. 74 API tests run hermetically; live mode is a config change, not a code change.

Reversible? Yes — the client is a thin seam; swapping in a provider SDK touches one module.

D-0012 — Web API access: `nginx /api/` proxy + demo auto-login client

Date. 2026-07-02 (P2 frontend + deployment)

Context. The Next.js app (:3000) and FastAPI (:8000) sit behind one `nginx` vhost (`5cb5f0620.abacusai.cloud`). Browser code needs a same-origin API base (no CORS) and a bearer token, but the demo deployment has no interactive login UI yet.

Decision. `nginx` routes `location /api/ { proxy_pass http://127.0.0.1:8000/; }` (prefix stripped) ahead of the catch-all `/ → :3000`. `src/lib/api/client.ts` resolves the base URL (env override `→ /api` in the browser `→ localhost:8000` for SSR/tests), auto-logs-in with the seeded demo user on first request, caches the JWT in `localStorage`, and retries once on 401. Both services run under `systemd` (`aether-api.service`, `aether-web.service`).

Alternatives. Next.js rewrites to :8000 (rejected: couples deploy topology into app config and double-proxies). Cookie session via NextAuth against FastAPI (deferred: real login UX is a later slice; the client seam already accepts an injected token).

Consequences. Same-origin API calls, zero CORS config, demo works logged-out; swapping auto-login for real auth only touches `getToken()`. **Reversible?** Yes.

D-0013 — Orchestration: `LangGraph StateGraph` with an inspectable per-node seam

Date. 2026-07-02 (P2-S10)

Context. Phase 2 needs the supervisor `→ scout → matcher → tailor → coverLetter` flow expressed as a real graph (the Phase-3 runtime target) while staying unit-testable without network or model calls, and while honouring the approval gate.

Decision. `AetherGraph` builds a compiled `@langchain/langgraph StateGraph` over an `Annotation`-typed state, and exposes `runNode(name, state)` — a direct, synchronous seam that records a `GraphRunRecord` per invocation. Approval-gated nodes (`tailor`, `coverLetter`) return `pending_approval` and halt the chain rather than acting. Tests exercise both the node seam and the compiled graph.

Alternatives. Hand-rolled async pipeline (rejected: throws away LangGraph checkpointing and interrupts we want in Phase 3). Only testing the compiled graph (rejected: per-node assertions and run auditing get much noisier).

Consequences. +6 TS tests; `@langchain/langgraph` + `@langchain/core` added to `packages/agents`. **Reversible?** Yes — nodes are plain functions; the graph wiring is one file.

D-0014 — Volatile OpenRouter free-tier model ids: env-configured with automatic fallback chain

Date. 2026-07-09 (post-review hardening)

Context. An independent review found the live LLM agent endpoints returning 500: three of the four configured OpenRouter free-tier model ids (`deepseek/deepseek-chat-v3-0324:free` , `qwen/qwen-2.5-72b-instruct:free` , `meta-llama/llama-3.1-8b-instruct:free`) had been retired upstream (HTTP 404). OpenRouter’s free-tier catalogue is volatile — ids appear and disappear without notice — so any hard-coded id will eventually rot.

Decision. Model ids stay configuration (`AETHER_MODEL_REASONING/STRUCTURED/FAST/LIGHT` env vars, refreshed to currently-live ids: `openai/gpt-oss-120b:free` , `qwen/qwen3-next-80b-a3b-instruct:free` , `meta-llama/llama-3.3-70b-instruct:free` , `meta-llama/llama-3.2-3b-instruct:free`) and `LLMClient` gains a resilience chain in `auto` mode: 1. call the configured model; 2. on any failure (404/429/5xx/network/empty content) retry once with `openai/gpt-oss-20b:free` ; 3. fall back to the recorded fixture if one exists; 4. otherwise raise a typed `LLMUnavailableError` , which the agents router maps to a clean **HTTP 503 “LLM backend unavailable”** — never an unhandled 500. Successful live calls record a fixture only when none exists, so curated replay fixtures are never clobbered by variable live output. CI and tests remain in `replay` mode.

Alternatives. Pinning to paid models (rejected: cost for a demo); querying the OpenRouter `/models` catalogue at startup (rejected: adds a network dependency to boot and still races mid-flight retirements); hiding failures by always serving fixtures (rejected: masks real outages).

Consequences. +7 pytest (`test_llm_resilience.py`) covering the fallback chain, fixture fallback and the 503 contract; `FabricationGuard` no longer false-positives on sentence-initial title-case words; the cover-letter agent gained a corrective drafting loop (≤ 3 attempts, feeding flagged terms back to the model). **Reversible?** Yes — model ids are env vars; the fallback chain is one method.

D-0015 — Tailoring guard: evidence normalization instead of raw-verbatim token matching

Context. In production the tailoring agent returned `changes: 0` on every run: the anti-fabrication validator in `resume_tailor.py` required every token of a rewritten bullet to appear verbatim in the original bullet set. Any legitimate rewording — a unicode hyphen (`e-commerce` vs `e-commerce`), `≈92%` vs `92 percent` , `10,000` vs `10000` , `leading` vs `led` , or a harmless stopword (“the”, “of”) — was flagged as fabrication, so the guard reverted every bullet and the agent could never produce a tailored resume.

Decision. Novelty is now judged against a normalized evidence index built from the full resume text (`_evidence_index` / `unsupported_tokens`):

1. **Unicode folding** — hyphen/dash/minus variants → `-`, curly quotes → `'`, `≈` / `~` → `~`, `%` → `%`, nbsp/thin spaces → space, `...` → `...`, `×` → `x`;
 2. **Case folding + light stemming** — suffix stripping (`ing`, `ed`, `er`, `est`, `es`, `ly`, `s`, `ies→y`) so inflectional variants of an evidenced word are accepted;
 3. **Number-format equivalence** — numeric tokens compare by value after comma/format stripping, so `92%`, `≈92%` and `92 percent` are one fact;
 4. **Stopword exemption** — function words and numeric qualifiers (`percent`, `approximately`, `roughly`, ...) carry no factual claim and are never counted as novel.
- A bullet is rejected (and reverted to the original) **iff** it still contains a content token whose stem/value is absent from the resume evidence — new skills, tools, employers and metrics are still rejected exactly as before.

Alternatives. Semantic-similarity scoring via the LLM (rejected: uses the very component the guard must not trust); whitelisting per-bullet diffs by hand (rejected: unscalable); loosening the guard to “reject only numbers” (rejected: would admit fabricated skills/tools).

Consequences. Tailoring now yields real accepted changes in production (verified live: changes: 22 with 3 genuinely-novel bullets rejected) while all negative fabrication tests still pass. +11 pytest (`test_guard_normalization.py`) pin the accept/reject contract.

Reversible? Yes — the normalization pipeline is three pure functions in `resume_tailor.py`.

D-0016 — Approval resolution propagates to the linked Application

Context. Phase-2 audit journey J4: approving or rejecting an `application_submit` approval flipped only the `ApprovalRequest` row. The linked `Application` (FK `applicationId`) stayed in `draft` forever, so the Applications kanban never reflected any decision (defect D2).

Decision. `ApprovalService.resolve()` now synchronises the linked application in the same operation: **approve** → **submitted**, **reject** → **rejected**. The update is guarded with `WHERE status = 'draft'` (and the owning `userId`) so a late decision can never regress an application that already advanced (e.g. to `interview`), and approvals without an `applicationId` are untouched.

Alternatives. A DB trigger (rejected: hides business logic in the schema); doing it in the router (rejected: the service is the single resolution path and is also used by tests/agents); an async job (rejected: adds a queue dependency for a two-row transaction).

Consequences. Approve/reject now has a visible, verified effect on the tracker — confirmed on production with DB before/after snapshots (draft→submitted and draft→rejected). +2 pytest in `test_approvals.py` pin the contract. **Reversible?** Yes — one static method call in `resolve()`.

D-0017 — Hard wall-clock LLM budget, shared pipeline deadline, and provider override

Context. Phase-2 audit defect D1: cover-letter and pipeline runs hit the edge’s ~100 s cut-off (524). `AgentRun` rows recorded coverLetter calls of 133–157 s despite a declared 60 s budget — httpx’s read timeout is **per chunk**, so a model that keeps trickling tokens never times out. In the pipeline, each agent’s own `LLMClient` had an independent budget, so budgets stacked.

Decision. Three changes in `llm_client.py` :

1. **Hard cap** — live calls execute in a worker thread; `future.result(timeout=...)` abandons any call that exceeds the wall-clock budget, regardless of streaming behaviour.
2. `shared_budget()` — a contextvar-based deadline; the pipeline wraps its `taylor+coverLetter` dispatches so all LLM clients in scope share one budget instead of adding theirs up.
3. **Provider override** — `AETHER_LLM_BASE_URL` / `AETHER_LLM_API_KEY` point the agent layer at any OpenAI-compatible endpoint (e.g. Anthropic's `/v1` with Claude model ids via `AETHER_MODEL_*`), and `AETHER_MODEL_FALLBACK` makes the fallback model configurable. OpenRouter remains the default when unset.

Alternatives. Raising the edge timeout (rejected: not under our control and hides the bug); async cancellation of the `httpx` stream (rejected: cancellation points depend on the server yielding); background jobs + polling (valid future work, but a UX/API contract change out of audit scope).

Consequences. On production, cover-letter runs complete in ≈ 60 s (HTTP 200) and the full pipeline in ≈ 62 s — no 524s. The abandoned worker thread may linger until the provider closes the socket (bounded, no user impact). +3 pytest in `test_llm_resilience.py`.

Reversible? Yes — env vars default to previous behaviour; the cap is one executor block.

D-0018 — Root-resume ingestion endpoint + explicit resume selection for tailoring

Context. Section C of the Phase-2 audit required registering a second base resume (the BA/PO variant, `assets/resume/Vik_Resume_BA_Final.pdf`) in the app so it appears in Resume Studio, persists in the DB, and is tailorable. The API previously had no way to create a resume — the only root resume was the demo-seeded one — and `TailoringAgent.run()` always tailored against `ensure_base_resume()`, ignoring any other root.

Decision. Two additive API changes:

1. **POST /resumes** (`routers/resumes.py`) — creates a new **root** resume (no `parentId`) from `{label, raw_text, contact?, format_hash?}`. Sections are built server-side (`raw_text`, bullets via `extract_bullets`, `contact`); `format_hash` defaults to `sha256(raw_text)[:16]`; version comes from `repo.next_version(user_id)`. Returns 201 with the stored resume.
2. **resume_id selection** — `TailoringAgent.run(user_id, job_id, resume_id=None)`: when a `resume_id` is supplied (via the existing `POST /agents/tailor/run` body, whose `JobTargetRequest` gained an optional `resume_id`), the agent tailors against that resume (`get_by_id`, unknown id $\rightarrow$ 404); otherwise behaviour is unchanged (base resume).

Alternatives. Multipart PDF upload with server-side parsing (rejected for audit scope: heavier surface, the PDF already lives in the repo and `scripts/ingest_ba_resume.py` extracts its text deterministically); a `is_default` flag switch on resumes (rejected: changes existing tailor semantics; explicit per-run selection is safer and reversible).

Consequences. BA resume registered on production as a root resume (id `c57a44d136100943494554143`, version 15); a live tailoring run against it produced **20 accepted changes** with a child resume inheriting the parent's `formatHash`. +4 pytest in `tests/test_resume_ingest.py` (create, 422 validation, tailor-with-resume_id, 404 unknown id).

Reversible? Yes — both changes are additive; omitting `resume_id` reproduces prior behaviour.

D-0019 — Swarm-directive conformance: real auth gate, 13-item nav, AU\$ salaries, 30-min discovery timer

Date: 2026-07-12 · **Author:** Aether Delivery Agent (swarm directive execution) · **Status:** Adopted

Context. The aether-swarm execution directive audits production against its Section 5.4 success criteria. Five failed: SC-AUTH-03 (silent demo auto-login meant anyone with the URL saw the workspace — BUG-013), SC-CL-02/SC-CC-07 (Cover Letter Studio absent from the sidebar), SC-CL-01 (`/dashboard/cover-letter` fell through to the catch-all), SC-CC-05 (salaries rendered bare `$`, not `AU$`), and SC-JOB-10 (no 30-minute scheduled discovery).

Decision.

1. `getToken()` no longer auto-logs-in with demo credentials; an unauthenticated browser session is redirected to `/login` and the request fails 401. A client-side `AuthGuard` wraps the dashboard layout. The `/login` form still prefills the demo account (demo environment).
2. **D-0002 amended:** the primary sidebar is now 13 items — “Cover Letter Studio” (`/dashboard/cover-letters`) sits between Resume Studio and Story Bank. The 2026-07-01 rejection targeted a then-phantom screen; the workspace is now real.
3. `next.config.mjs` permanently redirects `/dashboard/cover-letter` → `/dashboard/cover-letters`.
4. Salary labels prefix `AU$` (or `US$` when `currency === "USD"`).
5. `aether-discovery.timer` (systemd, `OnCalendar=*:00/30`, logs to `/var/log/aether/discovery.log`) runs `scripts/discovery_cron.sh`: `login` → scout with the user’s saved target role/location → fit-scorer. Runs are verifiable as AgentRun rows.

Consequences. REQ-05’s “20 named agents” remains deliberately unmet at 7 agents: fabricating 13 non-existent agent cards would violate the directive’s higher-priority constraint 2 (no mock/simulated data in production UI). The registry shows exactly the agents that exist.

D-0020 — Agents-screen truthfulness: env-derived providers, planned cards, real cost attribution, real ATS panel

Date: 2026-07-12 · **Author:** Aether Delivery Agent (swarm directive execution, phase 2) · **Status:** Adopted

Context. The second swarm-directive audit found the Agents screen surfacing fabricated data: provider cards defaulted to “connected” with invented details (“Claude Pro · 45 messages remaining”, “\$12.40 credit remaining”) regardless of real credentials; 16 catalog entries with no backend showed status “active”; run costs were priced against catalog-“recommended” models the runtime never uses; the Test Run modal returned simulated “actual” figures (`est × 0.97`); and the workflow graph mislabelled real agents (storyExtractor as “Learning”, matcher as “Memory”). Resume Studio also never displayed an ATS score (SC-RS-05).

Decision.

1. Provider status/detail/models derive from the server env at request time (`_provider_env_state`): Anthropic = connected via the direct `AETHER_LLM_API_KEY` subscription token on `api.anthropic.com` (SC-AG-05 — never via OpenRouter); OpenRouter = connected-standby (key present); OpenAI/Gemini/Groq/Bedrock = unconfigured (no keys). A user override can only downgrade, never fabricate a connection; PUT returns 409 when marking a keyless provider connected. The stale fabricated “gemini connected” row was deleted.
2. Catalog entries without a backend are status “planned” (roadmap cards, dimmed, no toggle) —

21+1 named cards satisfy REQ-05's spirit while constraint 2 bars fake activity. A new "Story Extraction Agent" card owns the real storyExtractor; "Learning / Feedback" is honestly planned; "Orchestration" maps to the real supervisor node.

3. Run costs bill the model the agent ACTUALLY uses (tailor/coverLetter →

AETHER_MODEL_REASONING

= claude-fable-5 at \$10/\$50 per MTok; storyExtractor → STRUCTURED = claude-haiku-4-5 at \$1/\$5);

deterministic agents (scout, fitScorer, matcher, supervisor) record zero LLM spend. Test Run

"actual" figures come from the last real completed run (null if never run).

4. The workflow graph shows the real 7-agent topology in pipeline order.

5. GET /resumes/{id}/ats scores a version against its source job with the deterministic ATS engine; Resume Studio shows the breakdown panel for tailored versions (SC-RS-05).

Consequences / accepted deviations. SC-RS-06 (AI-Detection %) is NOT implemented — no honest measurement exists without a third-party detector, and an invented percentage violates constraint 2. SC-AG-08/REQ-10 (Langfuse) and Gmail OAuth (SC-EC-02+, SC-AUTH-04) remain blocked on

credentials/interactive consent. SC-ST-05 is partial: per-agent model preference persists but the runtime stays pinned to the validated Anthropic tiers.

Amended by DOC-K (2026-07-13, per GAP-P4-055 / candidate C-20). The claim in the paragraph

above — "the runtime stays pinned to the validated Anthropic tiers" — does not match reality and is corrected here. `_provider_env_state()` (`apps/api/app/routers/agents.py`) only inspects the documented per-provider keys (Anthropic/OpenRouter/OpenAI/Gemini/Groq/Bedrock); it has no visibility

into `apps/api/app/services/llm_client.py`'s `ABACUS_API_KEY` fallback path, which is what actually serves tailor/coverLetter/storyExtractor runs in this environment (live calls execute against Abacus-subscription-backed deepseek/qwen models, not Anthropic). The result is that every provider card can read "unconfigured" while runs are, in fact, being served — the panel does not reflect the credential path actually in use. The fix (GAP-P4-055) extends the provider-status source to detect and surface the Abacus subscription fallback honestly (e.g. a distinct "Abacus subscription (fallback)" status) instead of reporting it as no provider at all or misrepresenting it as pinned to Anthropic. The "never fabricate a connection" principle from this ADR's Decision §1 is unchanged — what changes is that the fallback path itself must now also be represented truthfully, not omitted.

D-0021 — Scout truthfulness: upsert refreshes are not "discoveries"; §10.2 letter format enforcement

Date. 2026-07-12 · **Status.** Accepted

Context. `JobRepository.create` upserts on (userId, sourceUrl) and always returns a row, and the scout counted every returned row as `persisted`. With the 30-minute discovery timer live, every run re-found the same 5 postings and the dashboard feed rendered "discovered 5 new roles" in perpetuity while zero rows were inserted (last real insert hours earlier) — fabricated-looking activity in the production UI (constraint 2). Separately, generated cover letters used the banned generic opener and lacked the §10.2 business-letter structure (date, addressee, Re: line, 3-paragraph body, sign-off with the candidate's real name).

Decision.

1. The upsert's RETURNING clause exposes (`xmax = 0`) AS "wasInserted" ; the scout counts a job as `persisted` only on a true insert and as `updated` on a refresh. `ScoutResult`, the

`/agents/scout/run` response and the AgentRun output carry both counts.

2. The dashboard feed describes a zero-insert run as “checked job boards — no new roles” (metric “N refreshed”) instead of “discovered 0/5 new roles”.
3. Cover letters are composed as full business letters (Melbourne-tz date, addressee block, Re: line, salutation, 3-paragraph body, “Sincerely, “).

The corrective drafting loop feeds back §10.2 structural violations alongside guard flags; the banned generic opener is stripped deterministically if it survives retries. The system-generated date and signer join the guard’s evidence corpus so ground-truth tokens don’t false-positive.

4. Resume tailoring preserves quantified outcomes (a rewrite that drops every metric from a quantified bullet is rejected → original kept), never reuses an evidenceRef, and re-tailors against the parent version’s stored bullets so `changes` matches the visible diff.

Consequences. Historic AgentRun rows keep the old inflated `persisted` counts (feed shows them as recorded); counts are honest from deployment forward.

D-0022 — Job discovery transparency: every job links to its original posting; “new today” counts from midnight

Date. 2026-07-12 · **Status.** Accepted

Context. The Jobs screen showed zero external links: `sourceUrl` was in every API payload but never rendered, so nothing on the page let the user reach the actual posting — the submit gate even instructed “complete the submission via the job posting link” while no such link existed. This made live-scraped jobs read as fabricated. An audit confirmed the underlying data is real: the Seek adapter live-scrapes via Firecrawl (sampled postings match), all non-seek URLs resolve with matching titles, and `/jobs/{id}/insights` derives from the real ATS engine. Additionally “N new today” was a rolling 24-hour count (33) while only 5 jobs arrived since midnight, and a “Pull from Story Bank” link pointed at the non-existent `/dashboard/story-bank` route (catch-all panel).

Decision. Render `sourceUrl` as an external link (new tab, `rel="noopener noreferrer"`) everywhere a job appears: job-card source chip, saved-card source chip, detail-panel “Sourced from” chip, a dedicated “View posting ↗” button beside Preview, the submit-gate warning and its confirmation state, and the dashboard opportunity cards. “New today” counts from local midnight. The Story Bank link targets the real `/dashboard/stories` route.

Consequences. None adverse; jobs without a `sourceUrl` degrade to the plain chip (none exist in production today — 0 null/empty/demo URLs).

D-0025 — Job Discovery: role/salary dropdown filters + bulk tailor/saved-tailor-all deferred to Phase 3+

Date. 2026-07-13 · **Status.** Accepted

Context. The Job Discovery wireframe (`design/screens/job-discovery.html`) defines eight filter/action elements. Four are already live (source dropdown jd29, remote toggle jd07, match slider, and bulk skip jd11). Four are absent from the current implementation:

- **Role dropdown filter** (WIRE-0033 / jd04) — a chevron button that would open a role-category picker.
- **Salary dropdown filter** (WIRE-0036 / jd06) — a chevron button for salary-band selection.
- **Bulk “Tailor & Apply” button** (WIRE-0040 / jd10) — in the select-all bar, triggers multi-job tailoring before bulk-apply.

- **Saved “Tailor & Apply all” button** (WIRE-0059 / jd43) — in the Saved tab, tailors all saved jobs in one action.

The Preview button (WIRE-0052 / jd33) is already implemented as a `<Link>` to Resume Studio at `/dashboard/resume?job={id}` and the Location filter (WIRE-0035 / jd05) has a working client-side text input.

Decision. Defer all four missing elements to Phase 3+ for these reasons:

1. **Role + Salary dropdowns** require server-side query-param support. The `GET /jobs` endpoint and `JobRepository.list_by_user()` accept only `status`, `source`, `saved`, and `sort` — there are no `role` or `salary` filter columns. Adding them means (a) a DB migration for role-category/salary-band columns or (b) full-text/range filtering with index support. Both paths are material scope increases beyond this fix cycle. The dropdown UI pattern itself (chevron button → popup menu with predefined options) is a reusable component not yet in the design system.
2. **Bulk Tailor & Saved Tailor-all** require multi-job tailoring orchestration. The current `POST /agents/tailor/run` handles one job at a time. Running N tailoring jobs and N apply operations in a single user-facing action needs a queue or concurrency model, progress feedback, partial-failure handling, and likely a new agent endpoint. This is explicitly scoped as a Phase 3 pipeline feature.

Alternatives. (a) Add client-side text-input filters for role/salary (like the existing `locationQuery`) — rejected because the wireframe shows dropdown selectors with discrete options, and a free-text filter misrepresents the design intent while providing a brittle UX (titles aren’t categorised). (b) Add “Tailor & Apply” as a client-loop over sequential API calls — rejected because N sequential LLM calls from a browser would timeout and provide no progress feedback, making the feature appear broken.

Consequences. The Job Discovery screen ships with 4 of 8 wireframe filter/action elements live. The four deferred elements are documented here with clear acceptance criteria so Phase 3 can pick them up without rediscovery. No code changes in this slice. **Reversible?** Yes — each element can be added independently when its backend dependency is ready.

D-0026 — Mobile responsive layout deferred to Phase 3+

Date. 2026-07-13 · **Status.** Accepted

Context. The design wireframes include two mobile-specific screens at 390×844 viewport — `design/screens/mobile-dashboard.html` (7 mobile-specific elements including top bar greeting, notification badge, 2×2 stats grid, approval banner, activity feed with agent cards, and bottom tab bar) and `design/screens/mobile-approval.html` (9 mobile-specific elements including back button, centered approval header with counter, action card with confidence meter, AI reasoning checklist, trust-agent checkbox, and sticky bottom action footer). The current frontend implementation is desktop-first: the 13-item sidebar layout works at ≥1024px, a basic `MobileTabBar` provides 5-tab bottom navigation below that breakpoint, but no other screen has been adapted for mobile viewport. A structural audit against the mobile wireframes identifies 16 mobile-specific elements that are absent from the current build — the dashboard still renders its desktop card grid on a 390px viewport, and the approval flow shows the desktop modal rather than the dedicated mobile approval screen.

Decision. Full mobile-responsive layout — including the dedicated mobile dashboard, mobile approval screen, and responsive adaptations for all 13 workspace sections — is deferred to **Phase 3+** (post-MVP). Phase 2 ships with the desktop layout as the only supported viewport. The existing `MobileTabBar` and the responsive breakpoints already in the `ApprovalModal` footer (desktop row layout → mobile stacked layout) are kept as forward-compatible scaffolding, but no further mobile work is undertaken in Phase 2.

Rationale.

1. **No mobile user stories in Phase 2 scope.** The delivery brief targets a desktop web application; the mobile wireframes were produced as part of the original design exploration but were never in the Phase 1–2 delivery contract.
2. **The desktop layout is functional end-to-end.** All 13 workspace sections, the approval gate, the agent pipeline, and the job-discovery flow work correctly at $\geq 1024\text{px}$. Adding mobile breakpoints to every screen would be a non-trivial cross-cutting effort (~ 16 screens $\times$ responsive adaptation) that would delay Phase 2 completion without adding user-facing functionality.
3. **The scaffolding stays.** The `MobileTabBar`, the modal's responsive footer, and the `lg` breakpoint on the desktop sidebar are already wired and tested — they provide a clean seam for Phase 3 mobile work without bit-rot.

Alternatives. (a) Implement full mobile responsive in Phase 2 (rejected: out of scope, slows delivery, no mobile user stories). (b) Remove the `MobileTabBar` and responsive scaffolding entirely to avoid confusion (rejected: more work to remove than to leave; the scaffolding is tested and documents the intended mobile direction). (c) Ship a half-finished mobile experience where only the dashboard is adapted (rejected: worse than no mobile — inconsistent UX across sections).

Consequences. The 16 mobile-wireframe elements (7 dashboard + 9 approval) remain unimplemented and are tracked as a Phase 3+ backlog item. The mobile wireframes (`mobile-dashboard.html` , `mobile-approval.html`) serve as the design target for Phase 3. The existing responsive scaffolding is explicitly forward-compatible, not a partial implementation. **Reversible?** Yes — Phase 3 can pick up the mobile workstream by implementing responsive variants for each workspace section, reusing the existing `MobileTabBar` and sidebar breakpoint contract.

Amended by DOC-K (2026-07-13, per GAP-P4-031 / candidate C-31). The “remain unimplemented”

consequence above is now stale for half of this ADR's scope. Run-3 Stage A evidence (`mobile-dashboard__wireframe_fidelity__20260713T121243Z.json`) shows the mobile dashboard at 390×844 now renders all 8 wireframe sections (topbar, notification button, main content, 2×2 stats grid, approval banner, agent activity feed, bottom navigation) with 0 missing / 0 degraded — this half of the deferral has since been implemented and this is an improvement to record, not a gap.

Mobile Dashboard is no longer deferred and is reclassified from this backlog to VERIFIED. **Mobile Approval remains deferred** exactly as originally decided — the mobile-approvals sweep in the same run confirms 0/9 elements match (the desktop approval modal still renders at mobile viewport). The remaining Phase 3+ backlog under this ADR is therefore scoped to the mobile approval screen only (9 elements), not 16. GAP-P4-017 (which cited this ADR to close both halves) is corrected accordingly in the gap ledger.

D-0027 — Approval queue list view is intentional simplification of modal wireframe

Date. 2026-07-13 · **Status.** Accepted

Context. The design wireframe `design/screens/approval-modal.html` presents the approval flow as a modal dialog (560px wide, glass-raised panel, backdrop blur) containing a header with close button, action summary with confidence meter, “why approval is needed” panel, AI reasoning checklist, cover letter preview, trust-agent checkbox, and a footer with Reject / Edit & Approve / Approve buttons. The approval wireframe is designed as if the modal is the primary approval interaction — the user is expected to encounter one approval at a time, in a focused modal context.

The current implementation deliberately diverges: the primary approval interaction is a **queue list view** (`/dashboard/approvals`) showing all approval requests as cards with inline approve/reject buttons, status badges, confidence scores, expiry indicators, and a preview snippet. A “Review” button on each card opens the `ApprovalModal` (which faithfully implements all wireframe elements: close, Reject, Edit & Approve, Approve, plus mobile-stacked footer), but the modal is a secondary detail view — the queue list is the default interaction.

Decision. The list-first design is an intentional simplification of the modal-centric wireframe, for the following reasons:

1. **Batch efficiency.** A human reviewer typically handles multiple pending approvals in a session. A list view lets them scan all pending items at once, spot the highest-confidence or most-urgent approvals, and decide inline (approve/reject directly from the card) without opening a modal for each one. The modal wireframe’s one-at-a-time flow would require opening → reviewing → deciding → closing → repeating for each item.
2. **Status visibility.** The list shows approved/rejected items alongside pending ones (filterable), giving a complete audit trail. The modal wireframe has no concept of “previously decided” — it only shows the current pending request.
3. **The modal is preserved for deep review.** When detailed review is needed (reading the full cover letter, editing before approving, or reviewing AI reasoning), the “Review” button opens the full `ApprovalModal` with all wireframe elements intact. The `?review=<id>` deep-link parameter also lets any route (dashboard, notifications, email center) trigger the modal directly — the list is not the only entry point.

Wireframe parity. The `ApprovalModal` component implements 5 of 5 wireframe elements from `approval-modal.html`: close button (`btn-close-ap02`), Reject (`btn-reject-ap03`), Edit & Approve (`btn-edit-ap04`), Approve (`btn-approve-ap05`), and the full modal shell with backdrop. The 5 “missing modal elements” reported in the P4-018 audit refer to the fact that these elements are not visible on the list page itself — they live in the modal, which is one click away. This is by design: the list optimizes for scanning and fast decisions; the modal optimizes for detailed review.

Alternatives. (a) Make the modal the only approval interaction, removing the list (rejected: worse UX for multi-approval sessions; no audit trail visibility). (b) Show the modal on page load for the first pending approval (rejected: modal-on-load is intrusive and breaks the user’s orientation — the list provides context first). (c) Inline all modal elements into each list card (rejected: each card would be ~400px tall; the list would become unscannable).

Consequences. The approval flow has two tiers: **list for triage** (inline approve/reject, status filter, confidence-at-a-glance) and **modal for deep review** (edit, full reasoning, letter preview). The 5 wireframe modal elements are present in the `ApprovalModal` and reachable via the “Review” button or `?review=` deep link. The 48h expiry, trust-agent preference, and `DecisionContext` propagation work identically from both the list and the modal. **Reversible?** Yes — the list is one page component; replacing it with a modal-first flow in a future phase would not affect the `ApprovalModal` component, the API, or the approval service.

D-0028 — Wireframe mock/example numbers are design-time illustrations, not a production data contract

Date: 2026-07-13 · **Author:** DOC-K (T2, Phase 4 Run-3 doc rulings) · **Status:** Adopted

Context. Run-3 Stage A evidence found three screens where live production numbers differ from wireframe mock figures: (1) the Analytics/Dashboard funnel shows live Applications=13-16, Interviews=0, Offers=0, Jobs Found≈136-149 for the authenticated account, diverging from D-0003’s canonical set (847→412→156→23→4) (candidate C-14); (2) the Networking CRM stat tiles show live 1 contact / 0 active conversations / 0 referrals / 0% response rate versus the wireframe’s mock 48/12/5/41% (candidate C-21); (3) the Story Bank stat tiles show live 23 stories / 22 quantified versus the wireframe’s mock 24/19/11/94% (candidate C-29). D-0003’s original text states “these exact numbers must be used everywhere funnel data appears... deviations are bugs,” which, read literally, would make correct live per-user data itself a bug.

Decision. Wireframe stat-tile and funnel numbers (D-0003’s 847/412/156/23/4 funnel, the networking CRM mock 48/12/5/41%, the story-bank stat mock 24/19/11/94%) are design-time illustrative examples used to validate layout, typography, and visual hierarchy during design review — they are not a literal data contract the running application must reproduce. Production screens must display live, per-user data derived from real DB queries; a screen showing correct live numbers that differ from a wireframe’s illustrative mock is explicitly **not** a gap. D-0003 is amended accordingly (see its own amendment note): its binding requirement narrows to “every screen showing the funnel must compute it from the same live query” (already true — the Dashboard, Application Tracker, and Analytics screens all call the shared analytics/funnel endpoint), not “must literally read 847/412/156/23/4.”

Alternatives. (a) Keep D-0003 literal and treat every live-data screen as broken until it matches the mock numbers (rejected: would require hardcoding fabricated metrics into production, directly violating the §6/§8 ban on hardcoded/fake data — fabricating “847 jobs found” for an account that has actually found ~140 is a worse defect than the one being chased). (b) Silently leave the contradiction in D-0003 (rejected: an ADR that reads as binding will keep generating false-positive gaps in every future audit cycle).

Consequences. Candidates C-14, C-21, C-29 are closed as no-gap and recorded in the Verified No-Gap Register (Section E) with this ADR as the reason. Future audits should treat wireframe stat/funnel figures as illustrative unless a screen is caught computing a number incorrectly (a genuine metric bug — e.g. mislabeled source counts or a wrong per-week divisor, GAP-P4-058/059 — remains a real gap; this ADR does not blanket-cover computation errors, only value divergence from the mock). **Reversible?** Yes — this is a clarifying amendment; no code changes.

D-0029 — Email send: honest degraded UX when no provider is connected

Date: 2026-07-13 · **Author:** DOC-K · **Status:** Adopted

Context. Run-3 audit (candidate C-03; prior ledger GAP-P4-021 and GAP-P4-029, both duplicates of this one defect) found `POST /workspaces/emails/send` returns `200 {"status": "sent"}` unconditionally — the handler only appends to `EmailThread.messages` JSONB; no SMTP/Gmail/provider call exists anywhere in the repo, and the account's email provider status is `not_connected`. D-0020 established “never fabricate a connection” for the Agents-screen provider cards, but its text is scoped to that screen and does not reach the email-send code path, leaving this a live gap in the Email Center.

Decision. `POST /workspaces/emails/send` must return an explicit error — not `"sent"` — whenever no email provider is connected, mirroring the existing `PUT /providers/{id}` 409 pattern from D-0020 (e.g.

`409 {"error": "no_email_provider_connected", "message": "Connect an email provider to send. No email has been sent."}`). The Email Center UI must surface this as a visible error state, not a silent failure or an inferred success toast. Drafting, saving, and listing drafts (`POST /emails/draft` and the GET endpoints) are unaffected — only the send action changes. A real “sent” response is only permitted once a genuine provider integration (SMTP/Gmail OAuth/etc.) exists and the call actually succeeds. This closes GAP-P4-021 and GAP-P4-029 as duplicate reports of a single defect, now tracked under GAP-P4-042.

Alternatives. (a) Wire a stub/queued provider path that reports success (rejected: still fabricates completion of an action nothing performed). (b) Silently drop the send with no error (rejected: the user has no way to know the message never left the system — worse than an explicit error). (c) Block the Send button client-side only when no provider is connected (rejected: does not fix the API contract itself, and any other caller of the endpoint would still receive a fabricated “sent”).

Consequences. Until a real provider integration lands, every send attempt without a connected provider fails honestly and visibly; fabricated “sent” success is forbidden. GAP-P4-042 (FIX-C) implements the 409 response and the UI error surface. **Reversible?** Yes — the guard is additive to the send handler; a future real integration simply replaces the error branch with the actual send call.

D-0030 — Avatar management deferred: no backend storage exists

Date: 2026-07-13 · **Author:** DOC-K · **Status:** Accepted

Context. Candidate C-26 (GAP-P4-064) found the Settings → Profile section's wireframe “Change Avatar” control (`btn-avatar-st08`, “PNG or JPG, max 2MB”) absent from production. There is no avatar/file-storage backend anywhere in the API — no upload endpoint, no object-storage wiring, no `avatarUrl` column on the user model — so the control has no functional backend to attach to.

Decision. Avatar management (upload, storage, and display of a user profile photo) is deferred to Phase 3+. The Change Avatar button's omission from the Settings Profile section is accepted as consistent with current backend scope, not a regression or an oversight. Implementing it would

require net-new backend surface: a file-upload endpoint, size/type validation, object storage (or DB blob) wiring, and a schema column to persist the reference — none of which exist today.

Alternatives. (a) Render a disabled/greyed control with a “coming soon” affordance (a reasonable future micro-improvement per the D-0020 disabled-control-visibility principle, not required now — a fully-absent control is no worse than a disabled one for a feature with zero backend). (b) Implement a minimal upload path in this cycle (rejected: net-new storage dependency, out of Phase 4 fix-cycle scope).

Consequences. GAP-P4-064 is VERIFIED-CLOSED as documentary — this ADR is the evidence. The Settings Profile section ships without avatar upload; the wireframe control is tracked here as a Phase 3+ backlog item. **Reversible?** Yes — purely additive feature; no existing behavior changes.

D-0031 — Career-data consolidation scope: portfolio + GitHub ingested for real; LinkedIn limited to workspace-stored profile data

Date: 2026-07-13 · **Author:** DOC-K · **Status:** Adopted

Context. Candidate C-07 (GAP-P4-047) found the tailoring pipeline never consolidates GitHub/LinkedIn/portfolio content into rewrite context: `scrape_github_profile()` has zero callers app-wide, the “Portfolio Sync Agent” catalog entry is `status: planned` with `backend=None`, and `GET /settings` always returns a null portfolio block (`{url: null}`). The tailoring audit (clause a) expects rewrites to draw on the user’s full career-data profile, including portfolio and GitHub signal.

Decision. Career-data consolidation scope for this delivery cycle:

- Portfolio and GitHub are ingested for real.** `scrape_github_profile()` is wired into the tailoring context-build step (GAP-P4-047, FIX-J) so public repo/profile signal (languages, pinned repos, README summaries) joins the evidence corpus the fabrication guard checks against, and the Settings portfolio block persists and returns a real `url /sync` status once configured.
- LinkedIn is explicitly limited to workspace-stored profile data.** There is no LinkedIn API integration — LinkedIn does not offer general-purpose profile scraping to third-party apps without a partnership — so LinkedIn content only enters the system if the user pastes or stores it directly in their workspace profile. No live LinkedIn scraping or OAuth exists or is planned for this phase. This is an honest, documented limitation, not a silent gap: the Settings/Career Data UI must label LinkedIn input as “paste your LinkedIn summary” rather than implying an automatic sync.

Alternatives. (a) Scrape LinkedIn without authorization (rejected: violates LinkedIn’s terms of service and is not a defensible engineering position). (b) Defer portfolio/GitHub consolidation alongside LinkedIn (rejected: the GitHub scraper already exists in the codebase with zero callers — wiring it up is genuinely buildable now, so deferring it too would under-deliver relative to what’s actually available).

Consequences. GAP-P4-047 implements real portfolio+GitHub wiring into the tailoring context; LinkedIn remains workspace-paste-only with clear UI labeling — closing the tailoring audit’s clause (a) expectation for the two sources that can honestly be automated, while documenting the LinkedIn constraint as a scope decision rather than a defect. **Reversible?** Yes — GitHub wiring is additive

to the context-build step; a LinkedIn API integration can be added later without changing the workspace-paste path.

D-0032 — Interview Center: Phase 3+ deferral made explicit

Date: 2026-07-13 · **Author:** DOC-K · **Status:** Accepted

Context. Candidate C-27 (GAP-P4-065) found the Interview Center (`/dashboard/interviews`) renders only an empty-state placeholder (“No interview scheduled” + View Applications button) against a wireframe defining 26 elements (tabs, company brief, predicted questions, live-assist metrics, debrief) — only 4 (15%) implemented. A code comment in `apps/web/src/app/dashboard/interviews/page.tsx` already states “Interview Center — deferred to Phase 3+ ... No backend routes exist yet,” but no ADR named this deferral the way D-0025, D-0026, and D-0027 name theirs — only D-0009’s generic “honest placeholder for any unbuilt route” pattern applied, which covers the mechanism but not this screen specifically.

Decision. The Interview Center’s full wireframe (tabs, company brief, AI-predicted questions, live-assist metrics, post-interview debrief) is formally deferred to Phase 3+, following the same pattern and rationale as D-0025 (job filters/bulk actions) and D-0026 (mobile layout): no backend routes exist for interview scheduling/prep content, and building them is a material scope increase (new data model, new agent, new UI surface) beyond this fix cycle. The current empty-state placeholder — which already exceeds a bare 404 or generic catch-all per D-0009 — is the accepted Phase 4 state. This ADR formalizes in writing what the code comment already asserted informally.

Alternatives. (a) Implement a minimal Interview Center now (rejected: no interview-scheduling backend or predicted-questions agent exists; this is net-new feature scope, not a fix). (b) Leave the deferral undocumented as only a code comment (rejected: undiscoverable to auditors and inconsistent with how every other Phase-3+ deferral in this codebase is tracked via a named ADR).

Consequences. GAP-P4-065 is VERIFIED-CLOSED as documentary — this ADR is the evidence. The 26-element wireframe (`design/screens/interview-center.html`) remains the Phase 3 implementation target. **Reversible?** Yes — purely documentary; Phase 3 can implement against the existing wireframe without further ADR changes.

D-0033 — Auth rate limiting is keyed on the request identifier, not the client IP

Date: 2026-07-14 · **Author:** FIX-BE-RL · **Status:** Accepted

Context. The first rate-limiter for `/auth/register` and `/auth/login` (`apps/api/app/rate_limit.py`) keyed on the client IP (`request.client.host`). Review of the production topology found that key is not trustworthy here: the API runs behind `Envoy -> nginx -> uvicorn`, `nginx` sets no `X-Forwarded-For`, and `uvicorn`’s `ProxyHeadersMiddleware` trusts the loopback peer. That leaves two failure modes with no safe middle ground — either the limiter honours a client-supplied `X-Forwarded-For` (so any caller mints a fresh bucket per request with a spoofed value and bypasses the limiter entirely), or it keys on the single trusted `127.0.0.1` `nginx` hop (so every user on the internet collapses into one global bucket — a low

ceiling there is a site-wide auth denial-of-service the moment normal traffic exceeds it). The prior test `test_rate_limit_not_bypassable_via_x_forwarded_for` gave false confidence: FastAPI's `TestClient` never runs uvicorn's proxy middleware, so it could not exercise the real IP-trust bug.

Decision. Rate limiting is keyed on the **normalized submitted request identifier** carried in the request body — never on the client IP. The client IP is not read for rate limiting.

- * `/auth/login` — key = normalized (trimmed + lowercased) submitted identifier (email or username).

Only FAILED attempts are counted; the default cap is 5 failures / 15 min per identifier, then 429. A successful login resets that identifier's counter, so a legitimate user is never locked out by their own earlier typos. Different identifiers have independent buckets.

- * `/auth/register` — key = normalized submitted email. Every attempt counts; the default cap is 3 / hour per email, then 429 (blunts re-registration spam against one address).

- * No low global bucket is used — a shared low ceiling would reintroduce the site-wide DoS above.

Both caps are tunable via env (`AUTH_LOGIN_MAX_FAILURES` , `AUTH_LOGIN_WINDOW_SECONDS` , `AUTH_REGISTER_MAX` , `AUTH_REGISTER_WINDOW_SECONDS`). The limiter is an in-process, thread-safe sliding window (no new dependency). 429 responses carry a clear JSON message and a `Retry-After` header. Because the identifier travels in the JSON body (not the transport), this design IS fully exercisable through `TestClient` , so the tests assert the real behaviour.

HONEST RESIDUAL (limitation, not a defect). Identifier-keying stops brute-force against a single account and re-registration spam against a single email. It does **NOT** stop a distributed mass-registration attack that uses many different email addresses (each fresh email is a fresh bucket), nor distributed credential-stuffing that spreads guesses across many distinct accounts. Defending against those requires a trustworthy client-IP signal, a CAPTCHA / proof-of-work challenge, or email-verification before an account is usable — none of which exist in this deployment today. This ADR explicitly does **not** claim any IP-based or distributed-abuse protection; that gap is accepted for now and left to a future phase.

Alternatives. (a) Keep IP-keying with a hardened proxy chain (configure nginx to set a real `X-Forwarded-For` and uvicorn `forwarded_allow_ips` to trust only Envoy) — rejected for now: it depends on infra changes outside this repo and the reviewer verified the current chain does not provide it, so shipping IP-keying would be security theatre. (b) A low global bucket as a backstop — rejected: it is a user-visible DoS. (c) Add a CAPTCHA / email-verification to close the distributed residual — deferred: material new scope (new UX, provider integration) beyond this fix.

Consequences. `apps/api/app/rate_limit.py` is reworked to two identifier-keyed limiters wired in `app.main.create_app` and enforced inline in `apps/api/app/routers/auth.py` ; `tests/test_auth.py` replaces the misleading `X-Forwarded-For` test with honest body-keyed tests (per-identifier login lockout + reset, per-email register cap, and case-insensitive keying). **Reversible?** Yes — the limiter is in-process and self-contained; caps are env-tunable and a future IP/CAPTCHA layer can be added alongside it without changing the identifier-keyed core.

D-0034 — ADR-P6-SEEK: Seek scraping is ToS-prohibited; sourcing volume moves to licensed/official APIs

Date: 2026-07-16 · **Author:** Phase 6 orchestrator (researcher + fixer-hard) · **Status:** Adopted

Context. Phase 6 PROBE-13 and a dedicated live-research pass (`uat/reports/evidence/phase6/seek-tos-check.md`)

established that automated scraping of seek.com.au is explicitly prohibited: Seek's Terms of Service clause 4(d) bans automated data gathering without written consent, and Seek's `robots.txt` blocks `*/job/` and `/api/jobsearch/` and names `anthropic-ai` (alongside GPTBot/Bytespider) as disallowed agents. Production was, at the time, sourcing Seek jobs via a Firecrawl-based scraper — a live compliance

exposure, not a hypothetical one. Simultaneously, `GAP-P6-SRC-001` required restoring job-sourcing volume

(a pre-fix baseline of only 6 live jobs across 4 sources failed the ≥ 25 -job threshold).

Decision. Aether must **not** add to or continue automated scraping of seek.com.au. The `SeekAdapter` is moved behind a compliance gate (`AETHER_ENABLE_SEEK` , default **OFF**) and excluded from the live adapter registry (`apps/api/app/services/discovery/adapter_registry.py`) by construction — the scout

agent's live sync path cannot reach it unless the flag is explicitly set truthy. Sourcing volume is restored instead through **ToS-compliant sources only**: Adzuna AU (a licensed aggregator API, optional

`ADZUNA_APP_ID / ADZUNA_APP_KEY`) plus official ATS job-board APIs (Greenhouse, Lever, Ashby, Workable)

and public aggregators (Remotive, RemoteOK). Historical Seek rows already in the database are retained

(never deleted) but are excluded from every user-facing “active feed” list by a display-time freshness/source filter (`GAP-P6-DATA-001`), so stale/non-compliant data is never shown as live.

Alternatives. (a) Obtain Seek's written consent for automated access — rejected: no such consent exists today and pursuing it is a business/legal process outside this engineering effort's scope or timeline. (b) Keep scraping but reduce volume/frequency to “look” more compliant — rejected: the ToS prohibition is categorical (any automated gathering without consent), not a rate-limit question; a smaller violation is still a violation. (c) Drop Seek entirely (delete historical data too) — rejected: unnecessarily destructive; retaining history with a display-time filter satisfies both compliance (nothing new is scraped, nothing stale is shown as live) and non-destructiveness.

Consequences. `GAP-P6-SRC-001 / GAP-P6-SRC-002 / GAP-P6-DATA-001` are VERIFIED-CLOSED: a live production re-probe after this fix shows 30 active-feed jobs across 5 compliant sources (up from the 6-job/4-source baseline), 100% fresh within 30 days, 0 duplicate URLs, 0 Seek rows in the active feed —

while 149 legacy Seek rows remain retrievable via an explicit `include_stale=true` query, proving the filter does real work rather than coincidentally passing on empty data. The compliant-source margin is real but not large (documented honestly in `docs/delivery/PHASE6-BLOCKED-ON-HUMAN.md` 's GATE-07 risk note) — Adzuna credentials, when supplied by an operator, would add independent source diversity.

Reversible? Yes — `AETHER_ENABLE_SEEK` is a single env flag; re-enabling Seek (if a consent arrangement is ever reached) requires no code change, only a documented compliance decision.

D-0035 — ADR-P6-OAUTH: Anthropic third-party subscription OAuth is prohibited; API-key-only enforced

Date: 2026-07-16 · **Author:** Phase 6 orchestrator (researcher) · **Status:** Adopted — reverses the prior Phase-5 “USER-GATED, pending human OAuth consent” framing for this feature.

Context. Earlier phases (see the repo-root `EXECUTION-REPORT.md` , GATE-03) treated an Anthropic subscription OAuth connection (letting a user's own Claude Free/Pro/Max subscription pay for LLM calls, via PKCE) as a legitimate feature blocked only on a human clicking through a real OAuth consent screen.

Phase 6 commissioned a dedicated live-research verification

(`uat/reports/evidence/phase6/anthropic-oauth-verification.md`)

of whether this is actually supported. It is not: `platform.claude.com` 's auth documentation lists only API keys and Workload Identity Federation for third-party integrations; Anthropic's published Consumer Terms of Service (as of Feb 2026) explicitly prohibit using a Free/Pro/Max subscription OAuth token in any product, tool, or service other than Anthropic's own first-party clients (including via the Agent SDK); and the API has server-side blocked tokens matching the `sk-ant-oat01-*` prefix since 2026-01-09.

Decision. Building or shipping an active third-party consumer-subscription-OAuth flow would have Aether facilitate a Terms-of-Service violation against its own users and against Anthropic. Therefore: API-key authentication is the **only** supported and enforced production path for Anthropic-model calls (routed through OpenRouter per the existing runtime-provider architecture, `D-0014 /probe-03`). Any OAuth-flavored code in the repository stays behind `AETHER_ANTHROPIC_OAUTH_ENABLED=false` and the UI surfaces it, if at all, only as a disabled "coming soon — pending official Anthropic support" affordance — never as a clickable, functioning consent flow. GATE-04 (Phase 6's numbering; historically GATE-03 in

the Phase-5 report) is **CONDITIONALLY-CLOSED** with the verification artifact as evidence; it is explicitly **not** a human-gated item awaiting consent — no human action is being awaited on it at all, because the feature must not be built as active.

Alternatives. (a) Ship it anyway and let the human decide whether to click through — rejected: this platform does not knowingly build features whose only live-verification path requires a user to violate a third party's terms of service. (b) Wait and re-check periodically in case Anthropic changes policy — partially adopted: the flag-gated stub can be flipped on without a redeploy if/when official support exists, but no engineering effort is spent building toward it now beyond keeping the stub honestly disabled.

Consequences. This **reverses** the Phase-5 `EXECUTION-REPORT.md` 's framing of GATE-03/"Remaining User Action #1" (register an OAuth client, click "Connect with Anthropic") — that action item is withdrawn and marked, in that report, as something a human must **not** attempt. Billing separation remains absolute regardless: `claude-*` model calls only ever route through a configured credential (API key today), never a consumer subscription token, so quota/spend-cap accounting is unaffected.

Reversible? Yes — flipping `AETHER_ANTHROPIC_OAUTH_ENABLED` requires no schema or architecture change, only a policy decision if Anthropic later publishes official third-party OAuth support.

D-0036 — ADR-P6-STRIPE-MOCK: build billing now against a mocked Stripe SDK; live round-trip gates are human-gated

Date: 2026-07-16 · **Author:** Phase 6 orchestrator · **Status:** Adopted

Context. Subscription billing (`GAP-P6-BILL-001 / 002` , `GAP-P6-PRICING-001`) requires a real Stripe account, test-mode API keys, a registered webhook endpoint + signing secret, and Product/Price

objects to

genuinely exercise checkout → webhook → entitlement. None of these can be created by an autonomous agent

(they require a human to sign up for/administer a Stripe account) and the platform's own \$0.5 rule prohibits ever faking a live invoice or webhook event to manufacture a gate closure.

Decision. All billing code — schema, quota reserve/refund, GST math, webhook signature verification and in-transaction idempotency, checkout create-or-reuse-customer logic, the `/pricing` page — is built

and unit-tested now against a **mocked** Stripe SDK (dependency-injected `stripe_gateway`), so the entire

system is provably correct in isolation (webhook ordering, idempotency-on-replay, atomic quota reserve-then-refund, exact GST figures) without ever touching a real Stripe endpoint. The live round-trip gates that require an actual Stripe account (`GATE-13` checkout, `GATE-14` webhook, `GATE-15`

GST-on-invoice/Stripe-Tax, `GATE-16` cancellation, `GATE-33` webhook-signature-in-production, `GATE-34` backfill) are explicitly marked **BLOCKED-ON-HUMAN** in `docs/delivery/phase6-gap-analysis.json` and

`docs/delivery/PHASE6-BLOCKED-ON-HUMAN.md`, and are never closed by inference — `GATE-34` (backfill) is

the one exception verifiable without live Stripe, and was independently confirmed on production (2 existing users backfilled to 2 Subscriptions + 2 UsageQuota rows, additive/idempotent).

Alternatives. (a) Fabricate a fake Stripe webhook payload signed with a dummy secret to “prove” the live path — explicitly rejected: this is exactly the prohibited pattern of faking a live invoice/webhook event, and would produce false confidence about a code path (real signature verification against Stripe's actual signing algorithm and key format) that was never truly exercised. (b) Defer all billing code until a human provides Stripe credentials — rejected: this would block Cluster D and everything that depends on its schema (admin panel spend/quota views) for no benefit, since the mocked-SDK approach

already proves every piece of logic that doesn't require an actual external Stripe account.

Consequences. `GAP-P6-BILL-001 / 002 / GAP-P6-PRICING-001` are `FIX-READY-MERGED`: code complete, reviewed (`uat/reports/evidence/phase6/review-billing.json`, verdict PASS), 18 backend + 3 frontend tests

green, deployed to production (`/pricing` returns 200, billing schema materialized, `GATE-34` backfill verified) — but not `VERIFIED-CLOSED`, because the live Stripe round-trip has not actually happened. Closure of `GATE-13/14/15/16/33` requires an operator to supply Stripe test-mode credentials per `docs/delivery/PHASE6-BLOCKED-ON-HUMAN.md` item 1–4, after which a QA sub-agent re-runs the real checkout→webhook→cancel flow. **Reversible?** Yes — swapping the mocked SDK for the real one at that

point requires no application-logic change, only supplying the real credentials via environment variables.

D-0037 — ADR-P6-PRICING: ratified subscription tiers (binding — overrides the billing-architecture design doc's proposed quotas/annual pricing)

Date: 2026-07-16 · **Author:** Phase 6 orchestrator (fable-5), ratifying over `billing-arch` (opus) · **Status:** Adopted

Context. The `billing-arch` sub-agent's design (`docs/subscription/billing-architecture.md`) proposed monthly run-quotas of 50/200/600 and annual prices of A\$190/390/690, reasoning from its own unit-economics model. The originating prompt's §14.1, however, specified explicit tiers (A\$0/19/39/69 monthly, 30/100/300 runs), and Phase 6 STEP 3 commissioned live-verified cost/pricing research (`anthropic-pricing-verified.md` , `stripe-au-fees-verified.md` , `competitor-pricing-verified.md`) to check whether the prompt's figures were still defensible against current reality.

Decision. The live research **confirms** rather than contradicts the prompt's figures: verified Anthropic per-token pricing puts a Free-tier user's LLM cost at under A\$1.32/month (comfortably inside a A\$0 tier at 5 runs/month), and the resulting margins on GST-exclusive net revenue ($\text{net} = \text{total} \div 1.1$) range 82–89% across paid tiers; verified competitor pricing (7 competitors, live-fetched) places the proposed A\$19–69/month range solidly mid-market with a genuine agentic-automation differentiator no competitor offers. Per the platform's "maximum prompt execution accuracy" governance rule, an explicit prompt figure that is independently confirmed by live evidence is **ratified as-is**, overriding a sub-agent design's alternative proposal. The binding tiers:

Tier	Monthly (GST-inc AUD)	Annual (GST-inc AUD)	Agent runs/mo	Model-tier routing
Free	A\$0	—	5	light (Haiku-equivalent)
Starter	A\$19	A\$179	30	light + standard (Haiku + Sonnet-equivalent)
Pro	A\$39	A\$359	100	light + standard
Power	A\$69	A\$649	300	full model access

GST per the prompt's §14.2 formula: $\text{gst} = \text{round}(\text{total}/11, 2)$, $\text{net} = \text{total} - \text{gst}$ — e.g. A\$19 → gst A\$1.73 / net A\$17.27; A\$39 → gst A\$3.55 / net A\$35.45; A\$69 → gst A\$6.27 / net A\$62.73. Annual GST

uses the identical formula against the annual total. "Model tier" describes the app's own OpenRouter

model-routing policy per plan; it does not imply a direct Anthropic API/OAuth connection (see `ADR-P6-OAUTH`).

Alternatives. (a) Adopt the billing-arch design's 50/200/600-run, A\$190/390/690-annual proposal — rejected: it isn't wrong on its own unit-economics logic, but the prompt's explicit, now-confirmed figures take precedence per the governance rule, and adopting a different number without a documented

reason would itself be an undisclosed scope deviation. (b) Split the difference (e.g. average the two quota schemes) — rejected: arbitrary, and defeats the purpose of ratifying against verified evidence.

Consequences. `apps/api/app/repositories/billing.py` 's `RATIFIED_PLANS` seeds exactly these four tiers' prices/quotas/spend-caps (Free cap A\$1, Starter A\$5, Pro A\$15, Power A\$40 — admin-adjustable);

`test_plans_endpoint_is_public_and_returns_ratified_tiers` asserts all twelve headline numbers through

`GET /billing/plans` live. Stripe Tax is **off** at launch (its ~A\$140/month floor is disproportionate at launch volume; GST math is identical either way via the `round(total/11,2)` formula) — see

`docs/subscription/BILLING-ARCH-APPROVAL.md` . **Reversible?** Yes — pricing/quota changes are a `Plan` row update, not a schema change; Stripe Tax can be enabled later without touching the GST calculation.

D-0038 — Tailoring anti-fabrication: dedicated entailment-verification budget, separate from generation

Date: 2026-07-16 · **Author:** Phase 6 fixer-hard / reviewer (GAP-P6-TAIL-003/004) · **Status:** Adopted

Context. Resume tailoring already guarded against fabrication via keyword/evidence-token matching, but

a live QA round (`qa-prod-craft2.json`) found one unsupported claim (“for financial institutions”) slip through on an otherwise genuine lift. The robust fix — an LLM-based entailment-verification pass that checks each proposed bullet rewrite against the evidence corpus before accepting it — initially introduced a **new** regression: the entailment call shared the single `AETHER_LLM_BUDGET_SECONDS` wall clock with the tailor-generation call itself, so a slow generation call left the verifier 0–9 seconds — it timed out and, being fail-closed, conservatively reverted **every** proposed change, including genuinely evidence-backed ones (`qa-prod-craft3.json` : zero fabrication, but zero lift either, in 17/17 completions). A second contributing defect: `proper_noun_anchors()` / `_scoped_evidence()` treated generic

capitalized words (e.g. “Business”, “BI”) as context anchors, incorrectly excluding a story's own evidence from its home bullet.

Decision. Give entailment verification its **own** dedicated budget reservation

(`AETHER_LLM_ENTAILMENT_BUDGET_SECONDS`), carved out separately from the tailor-generation budget rather

than sharing one wall clock — so a slow generation call cannot starve the verifier to zero. Narrow the anchor heuristic to genuine proper nouns (employer/program names), excluding generic capitalized words,

so a bullet with no real anchors can draw on the full evidence corpus rather than being wrongly scoped

out. Zero-fabrication behavior is preserved (the entailment pass still reverts genuine fabrications fail-closed); this fix targets only the false-positive over-reversion and budget-starvation failure modes, per `review-tail4.json` (verdict PASS).

Alternatives. (a) Give entailment a fixed, generous budget regardless of batch size (e.g. flat 15s) — partially adopted as an intermediate step, but superseded by D-0039’s batch-size-scaled budget once the underlying “full-resume batch is too large to complete + verify under one edge timeout” cause was identified. (b) Drop entailment verification and rely solely on token-matching — rejected: token-matching alone already let one fabrication through; entailment is the more robust check §9’s zero-tolerance requires.

Consequences. Deploy configuration: `AETHER_LLM_BUDGET_SECONDS=65`, `AETHER_LLM_ENTAILMENT_BUDGET_SECONDS=15`, `AETHER_LLM_PRIMARY_BUDGET_FRACTION=0.3` (per reviewer recommendation in `review-tail4.json`). GAP-P6-TAIL-003 / 004 VERIFIED-CLOSED; lift restored without reintroducing fabrication risk, later refined further by D-0039. **Reversible?** Yes — both budgets are env-tunable; no schema change.

D-0039 — Tailoring top-K batch cap + scaled entailment budget: genuine lift within the synchronous HTTP edge

Date: 2026-07-16 · **Author:** Phase 6 fixer-hard / reviewer (GAP-P6-TAIL-005) · **Status:** Adopted

Context. Even with D-0038’s dedicated entailment budget, tailoring a full ~18-21-bullet resume in one LLM call was itself too slow (and the resulting large candidate set too slow to verify) under production’s fixed HTTP edge (~100 seconds, not under application control — see `DEFECTS-PHASE-2-AUDIT.md`

D1, `DECISIONS.md` prior discussion). This produced two failure modes depending on how the shared budget

was split: either the tailor-generation call was starved (16% completion rate observed in

`qa-prod-craft4.json`) or a genuinely evidence-backed batch of proposed edits ran out of entailment time

and was blanket-reverted (fail-safe correctly avoiding fabrication, but delivering zero measurable ATS lift — a value problem, not a correctness one).

Decision. Cap tailoring generation to the top- K (8) highest-impact/most-JD-relevant bullets per request instead of rewriting the entire resume, and scale the entailment-verification budget with the actual candidate count (`base_seconds + per_candidate_seconds × N`, capped) instead of a fixed budget. A

smaller batch both generates faster and produces a smaller entailment workload that reliably completes

within budget, so genuine lift can survive verification rather than being reverted for lack of time.

Zero-fabrication is preserved end-to-end — the entailment pass still reverts any genuinely unsupported

claim, fail-closed, regardless of batch size.

Alternatives. (a) Increase the overall wall-clock budget instead of capping batch size — rejected: the ~100s edge timeout is fixed and outside application control; a longer in-app budget only replaces

a

clean 503 with a raw edge-level gateway timeout on the slowest calls (this exact tradeoff is documented in

review-authenticity2.json 's deploy-budget-recommendation). (b) Cap the batch size but keep a fixed

entailment budget — rejected: a fixed budget either wastes headroom on small batches or still starves on

the largest allowed batch; scaling with candidate count is the more precise fix.

Consequences. Deploy configuration: `AETHER_LLM_BUDGET_SECONDS=65` ,
`AETHER_LLM_PRIMARY_BUDGET_FRACTION=0.4` , `AETHER_TAILOR_MAX_BULLETS=8` ,
`AETHER_LLM_ENTAILMENT_BASE_SECONDS=6` , `AETHER_LLM_ENTAILMENT_PER_CANDIDATE_SECONDS=2.5` ,
`AETHER_LLM_ENTAILMENT_MAX_SECONDS=25` (worst case ~90s, leaving ~10s edge margin — per
review-tail5.json). `GAP-P6-TAIL-005` `VERIFIED-CLOSED`: a fresh 10-attempt live production QA
round

(qa-prod-craft5.json) showed 8/10 honest completions (80%, up from 16% pre-fix), 2 with a genuine,

independently GET- /ats -reconfirmed lift (30.81→32.97), and **zero** fabrication survivors across all 8.

Accepted, documented residual: the remaining ~20% of attempts still return a clean HTTP 503 (the

honest `AETHER-AUTH-002` fail-closed path, never a fixture) because some LLM calls simply run longer than

the capped batch's budget even after this fix — this is a genuine synchronous-latency ceiling, not a correctness defect, and is tracked as `BACKLOG-P6-02` (durable fix: asynchronous submit→poll generation,

out of Phase 6 scope). **Reversible?** Yes — all four env vars are tunable; the batch cap and budget formula are config, not architecture, changes.

D-0040 — Networking workspace: single-page kanban, no UI delete, no LinkedIn import, stage-label mapping

Date: 2026-07-29 · **Author:** STAGE-C-FIX (G-P4-NETWORKING-WIREFRAME-005) · **Status:** Adopted

Context. The `design/screens/networking.html` wireframe shows a multi-tab workspace (Contacts, Outreach, Templates, Analytics tabs) with an “Import from LinkedIn” button, delete controls on contact cards, and a 5-column pipeline. The Phase 4 gap analysis (2026-07-26) flagged the networking screen as UNVERIFIED and noted it had no ADR coverage. ML-networking-001 (2026-07-22)

flagged the missing UI delete control as “UNSURE” and deferred adjudication to “Stage-3.” The screen has since been hardened with 10 Vitest tests (MV-networking-001 through -010) and the backend has full CRUD endpoints for contacts and outreach tasks. This ADR documents the intentional design decisions that differ from the wireframe.

Decision.

1. **Single-page kanban, not multi-tab workspace.** The wireframe's tabbed layout (Contacts / Outreach / Templates / Analytics) is replaced by a single-page layout where the pipeline, outreach queue, and communication log are all visible at once. This is the correct choice for a lightweight CRM: the screen's primary purpose is triaging contacts through pipeline stages, and a tabbed layout would hide the outreach queue and communication log behind clicks that

users would rarely make. The kanban-first layout puts the pipeline front and center, with the queue and log as companion panels. No multi-tab workspace is planned.

2. **No UI delete control for contacts (ML-networking-001).** The backend `DELETE /networking/contacts/{contact_id}` endpoint exists and works (returns 204, cascades to dependent outreach tasks). The UI intentionally does not expose a delete control. Rationale: (a) delete is a destructive, irreversible affordance — it needs a confirmation dialog, undo path, and/or trash/recovery pattern that would add complexity out of proportion to the screen's role; (b) contacts are intentionally cumulative — the CRM is a lightweight tracker, not a high-churn list; (c) the existing delete endpoint is available for admin/API cleanup and is tested. A UI delete affordance may be added in a future phase but is explicitly not required for correctness, honesty, or wireframe fidelity.
3. **No LinkedIn import (MV-networking-003).** The wireframe's "Import from LinkedIn" button implied a LinkedIn OAuth integration that does not exist. The implementation honestly relabels the control to "Add contact manually" and removes the LinkedIn branding. Building a LinkedIn OAuth import is a separate product decision (requires LinkedIn API access, OAuth consent flow, contact-mapping logic, and ToS compliance review) and is not committed for any phase. The empty-state copy was updated to remove the "Import your LinkedIn connections" claim (MV-networking-003 test asserts this).
4. **Stage-label mapping: DB enum → UI labels.** The `ContactStage` Postgres enum stores `identified | contacted | responded | meeting | referral`. The UI maps these to wireframe pipeline labels: `identified → New`, `contacted → Warm`, `responded → Active`, `meeting → Scheduled`, `referral → Placed`. The mapping is implemented in `apps/api/app/routers/workspaces.py` `networking_summary()` and is tested by `test_workspaces_networking_summary_buckets_real_contacts_into_pipeline`. The UI labels are simpler and more intuitive than the raw enum values; the enum values are kept as the canonical data model because they accurately describe the outreach lifecycle.
5. **Contact detail panel is read-only.** The detail modal (MV-networking-005, opened by clicking a contact card) displays the stored fields from `GET /networking/contacts/{id}` but does not offer inline editing. The backend `PATCH /networking/contacts/{contact_id}` endpoint exists and works, but wiring a full edit form is deferred. The detail panel satisfies the immediate need (seeing a contact's email, LinkedIn URL, and stage) without the complexity of an edit-save-cancel form.
6. **Outreach tasks are read-only in the UI.** The backend supports full CRUD for outreach tasks (`POST/PATCH/GET /networking/outreach`), but the UI renders the outreach queue and communication log as read-only cards. Creating and editing outreach tasks from the UI is deferred. The existing endpoints are consumed by the backend's own agent-driven workflows (e.g., the networking agent scheduling follow-ups) and are available for API consumers.

Alternatives. (a) Build the multi-tab workspace from the wireframe — rejected: adds navigation friction for no benefit; the single-page kanban is a better UX for the CRM's primary job (pipeline triage). (b) Add a delete button with a confirmation dialog — deferred: adds destructive-affordance complexity for a low-priority feature; the backend endpoint remains available for programmatic cleanup. (c) Rename the DB enum values to match UI labels — rejected: the enum values are semantically accurate (`identified`, `contacted`, etc.) and renaming a Postgres enum is a migration with no upside; the mapping is one dict in the router.

Consequences. The networking screen is now fully documented with an ADR. The 10 MV-networking- Vitest tests pass (all green). The screen's honest states (empty, single-contact, populated) are verified. ML-networking-001 is formally adjudicated as "intentional-by-design" with this ADR as the authority. GAP-P4-005 (networking-ADR gap) is closed. Reversible?* Yes — every decision above is a UI/UX choice, not a schema or architecture constraint. The backend endpoints are complete; the UI can add tabs, delete controls, edit forms, or LinkedIn import in a future phase without breaking any contract.

D-0041 — Part-2 remediation: approvals re-request affordance, networking delete UI, Market Pulse scope, admin CRON status source

Date: 2026-07-30 · **Author:** PART2-REMEDIATION (adversarial production review 2026-07-30) · **Status:** Adopted

Context. The 2026-07-30 adversarial production review (/home/ubuntu/aether_review_2026-07-30/ADVERSARIAL-PRODUCTION-REVIEW-2026-07-30.md) identified four items requiring a documented decision: (C-4) the approvals deadlock — applications flagged needs approval on the board while /dashboard/approvals shows "Queue clear" because requests expire after 48h with no recovery affordance; (ML-networking-001) the deferred contact-delete UI; (GAP-P4-004) Market Pulse "not connected"; and (C-6) the admin panel reporting CRON not_configured while a systemd discovery timer demonstrably fires every 30 minutes.

Decisions.

1. **Approvals re-request affordance (closes C-4 recovery gap).** The Applications board's "ready" column now distinguishes two states per card: a live pending approval renders the existing needs approval badge; a card whose approval has expired (or was never created) renders a no pending approval badge plus a **Request approval** button. The button POSTs the existing POST /approvals endpoint (type: application_submit , application_id , payload job_id/job_title/company), which is idempotent per (job_id, kind, pending) — double-clicks and races return the same pending approval rather than duplicates. Approving the re-requested item transitions the draft application to submitted via the existing _sync_application path. No auto-approval is introduced: the human stays in the loop; the fix only restores a path forward after expiry. Alternatives rejected: (a) non-expiring approvals — expiry is a deliberate safety property for an agent allowed to submit applications on the user's behalf; (b) auto-transitioning the application state on expiry — destroys user intent silently. Tests: apps/web/src/app/dashboard/applications/__tests__/page.test.tsx (badge/button/POST cases), apps/api/tests/test_part2_remediation.py::TestApprovalReRequestMigration (orphaned draft → re-request → idempotency → approve → submitted).
2. **Networking contact-delete UI (closes the D-0040 §2 deferral / ML-networking-001).** D-0040 §2 intentionally deferred a UI delete control. That deferral is now closed: the contact detail modal exposes a two-click-confirm **Delete contact** button (first click arms, second click calls the existing DELETE /networking/contacts/{id} ; arming resets when the modal switches contacts). This satisfies D-0040's own stated precondition for adding the affordance ("needs a confirmation dialog") with the minimal proportional pattern. Backend is unchanged (endpoint existed and was tested since 2026-07-22). D-0040 §2 is superseded on this

point only; all other D-0040 decisions stand. Tests:

`apps/web/src/app/dashboard/networking/__tests__/page.test.tsx` (arm/confirm/delete/refetch cases).

3. **GAP-P4-004 Market Pulse: declared out of scope for the current phase.** Wiring a real market-data source (salary benchmarks, demand indices) requires a paid external data provider (procurement/human-gated) and is not committed for any current phase. The Analytics screen keeps the honest “Market Pulse not connected” state — no synthetic or placeholder market data will be rendered. This ADR is the adjudication the review asked for (“wire a real market-data source or write the ADR declaring it out of scope”).

Reversible? Yes — the panel and its honest empty state remain; a data source can be wired in a future phase without contract changes.

4. **Admin CRON status derived from scout run recency (fixes C-6).** The deployment’s scheduler is a host-level systemd timer (`aether-discovery.timer`), invisible to the API process, so the previous hardcoded `not_configured` was false. The admin health endpoint now derives CRON status from the observable effect of the scheduler: `MAX("startedAt")` of `AgentRun` rows for the `scout` agent — `ok` if within 90 minutes (3× the 30-minute cadence), `stale` if older, `not_configured` if no runs exist, `error` on DB failure, with `lastRunAt` and a detail string. Alternatives rejected: (a) shelling out to `systemctl` from the API — couples the app to the host init system and fails in containers/tests; (b) an env flag declaring cron configured — reintroduces the exact class of asserted-not-observed status the review flagged. Tests: `apps/api/tests/test_part2_remediation.py::TestAdminCronStatus` (`not_configured/ok/stale`).

Consequences. The approvals pipeline no longer dead-ends after expiry; ML-networking-001 is closed as implemented (superseding its “intentional-by-design without UI delete” adjudication); GAP-P4-004 is formally out of scope with the honest banner retained; the admin panel reports scheduler health from evidence rather than assertion. All four changes are covered by `fails-before/passes-after` tests (evidence: `uat/reports/evidence/phase4/`).